

Java Backend Architecture

Interview Series

Chapter 19.5

AWS Advanced – Serverless & Edge

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.5 – AWS Advanced – Serverless & Edge

Overview

AWS Lambda with SnapStart enables near-instant Java cold starts, transforming Lambda from an impractical choice for Java to a competitive serverless option. API Gateway provides managed HTTP/REST/WebSocket APIs. CloudFront delivers content globally with edge caching and WAF integration. Understanding cold start mitigation, Lambda concurrency, and CloudFront cache behaviours is essential for modern Java cloud architectures.

Key Definitions

Term	Definition
Lambda	Serverless compute. Event-driven, auto-scaling, pay per 100ms. Java 21 with SnapStart.
SnapStart	Lambda feature for Java: restores from a post-init snapshot. Near-zero cold start (sub-second).
Cold Start	Lambda invocation latency when no warm execution environment exists. JVM init: 5-15s without SnapStart.
Provisioned Concurrency	Pre-warmed Lambda instances. Eliminates cold starts but costs money even when idle.
API Gateway REST API	Managed API: request validation, throttling, caching, API keys, usage plans.
API Gateway HTTP API	Faster, cheaper (70% less than REST). JWT authorizer built-in. Simpler feature set.
CloudFront	CDN with 400+ edge locations. Terminates TLS, caches responses, integrates WAF.
Lambda@Edge	Lambda functions running at CloudFront edge nodes. Modify requests/responses globally.
CloudFront Origin Access Control	Restricts S3 bucket access to CloudFront only. Prevents direct S3 URL access.
WAF	Web Application Firewall. Attaches to CloudFront/ALB/API Gateway. Blocks SQLi, XSS, rate limits.

Diagram 1: Lambda SnapStart Flow

WITHOUT SnapStart:

Invoke -> [Init phase: JVM start + Spring init] (5-15s) -> [Handler] -> Response
 |_____cold start penalty_____|

WITH SnapStart (Java 21):

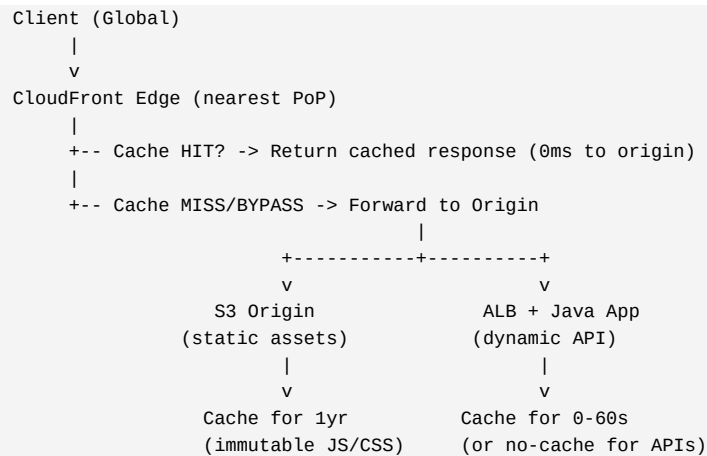
Deploy -> [Init once: JVM + Spring init + checkpoint snapshot]

Invoke -> [Restore from snapshot] (sub-second) -> [Handler] -> Response
 |___fast restore___|

Lambda SnapStart: takes snapshot after init, restores per invocation

Caveats: randomness/time from init must be refreshed (CRaC hooks)

Diagram 2: CloudFront Cache Behaviour



Code Examples

Lambda SnapStart (Java 21):

```
# SAM template
Resources:
  JavaApiFunction:
    Type: AWS::Serverless::Function
    Properties:
      Runtime: java21
      Handler: com.example.LambdaHandler::handleRequest
      SnapStart:
        ApplyOn: PublishedVersions
      AutoPublishAlias: live
      MemorySize: 1024
      Timeout: 30
      Environment:
        Variables:
          SPRING_PROFILES_ACTIVE: lambda
```

Java Lambda with Spring Cloud Function:

```
// Handler using Spring Cloud Function
@SpringBootApplication
public class LambdaApp implements ApplicationContextInitializer<GenericApplicationContext> {

    public static void main(String[] args) {
        SpringApplication.run(LambdaApp.class, args);
    }

    @Bean
    public Function<APIGatewayProxyRequestEvent, APIGatewayProxyResponseEvent> processOrder() {
        return request -> {
            Order order = parseBody(request.getBody());
            orderService.process(order);
            return APIGatewayProxyResponseEvent.builder()
                .statusCode(200)
                .body(toJson(result))
                .build();
        };
    }

    // CRaC hook: refresh state after SnapStart restore
    @Override
    public void initialize(GenericApplicationContext context) {
        // Re-generate random tokens, re-establish connections after snapshot restore
    }
}
```

CloudFront with ALB Origin:

```
# Terraform
resource "aws_cloudfront_distribution" "java_api" {
  origin {
    domain_name = aws_lb.java_alb.dns_name
    origin_id   = "JavaALB"
    custom_origin_config {
      http_port = 80; https_port = 443
      origin_protocol_policy = "https-only"
    }
  }
}

default_cache_behavior {
  allowed_methods      = ["GET", "HEAD", "OPTIONS", "PUT", "POST", "PATCH", "DELETE"]
  cached_methods       = ["GET", "HEAD"]
  target_origin_id     = "JavaALB"
  cache_policy_id      = "4135ea2d-6df8-44a3-9df3-4b5a84be39ad" # CachingDisabled for APIs
  origin_request_policy_id = "b689b0a8-53d0-40ab-baf2-68738e2966ac" # AllViewer
  viewer_protocol_policy = "redirect-to-https"
```

```
}  
  
web_acl_id = aws_wafv2_web_acl.main.arn    # WAF protection  
price_class = "PriceClass_100"  
}
```

Interview Q&A – AWS Advanced – Serverless & Edge

Q1. How does Lambda SnapStart eliminate Java cold starts?

SnapStart: after initial deployment, Lambda initialises the Java runtime (JVM start, Spring context init), then takes a memory+disk snapshot (CRaC checkpoint). On invocation, Lambda restores from snapshot instead of re-initialising -- sub-second restore vs 5-15s cold start. Caveat: network connections, random seeds, and time-sensitive state from init phase must be refreshed in CRaC beforeCheckpoint/afterRestore hooks.

Q2. When should you use API Gateway HTTP API vs REST API?

HTTP API: lower cost (70% cheaper), lower latency, built-in JWT/OIDC authorizer, Lambda/HTTP integrations. Missing: request/response transformations, API keys, usage plans, per-method throttling, caching. REST API: full feature set, request validation, caching, API keys, usage plans, mock integrations. For Java microservices: HTTP API for simple Lambda backends with JWT auth; REST API when you need caching, request validation, or granular throttling per method.

Q3. What is Lambda concurrency and how does reserved concurrency work?

Concurrency = simultaneous Lambda invocations. AWS account default: 1000 concurrent Lambdas per region. Reserved concurrency: allocate a fixed number of executions to a specific function (guarantees capacity + limits blast radius). Provisioned concurrency: pre-initialised environments at fixed cost (eliminates cold starts for critical Java Lambdas). Throttle a function: reserved concurrency = 0 (rejects all invocations, useful for circuit breaker pattern).

Q4. How does CloudFront reduce latency for global Java API clients?

CloudFront terminates TLS at the nearest of 400+ edge locations -- reduces TLS handshake RTT from 200ms+ (cross-continental) to < 20ms (edge). Caches static responses at edge (no origin request). For dynamic Java APIs: cache based on path pattern with short TTL (1-60s). Dynamic responses bypass cache but still benefit from persistent TCP connection from edge to ALB over AWS backbone (faster than public internet routing).

Q5. What is CloudFront Origin Access Control (OAC)?

OAC allows CloudFront to authenticate to S3 using SigV4 signing. S3 bucket policy restricts access to CloudFront's service principal only. Without OAC: S3 URLs are publicly accessible directly (bypassing CloudFront WAF, logging, CDN). With OAC: clients can only access content via CloudFront -- enforces WAF rules, centralized access logging, single cache-busting mechanism. Required for secure Java SPA deployments.

Q6. How do you handle Lambda throttling in Java event-driven architectures?

SQS trigger + Lambda: SQS acts as buffer -- if Lambda throttled, messages stay in queue with visibility timeout. Lambda service automatically reduces batch size. Dead-letter queue catches failures after maxReceiveCount. Kinesis trigger: Lambda retries from same shard position on throttle (at-least-once). EventBridge: has built-in retry policy. Monitor: concurrently_executions metric, Throttles count. Scale: increase reserved concurrency limit.

Q7. What are CRaC hooks in Lambda SnapStart?

CRaC (Coordinated Restore at Checkpoint): Java API for snapshot lifecycle. Resource implements org.crac.Resource: beforeCheckpoint() (called before snapshot -- close DB connections, flush caches, stop timers) and afterRestore() (called after restore from snapshot -- re-open connections, re-seed random, refresh credentials). Without hooks: restored Lambda may have stale DB connections, expired credentials, or predictable random seeds (security risk).

Q8. How does CloudFront WAF protect Java backends?

WAF attached to CloudFront distribution: rules execute at edge (400+ locations) before request reaches Java backend. Rule types: AWS Managed Rules (OWASP Top 10, known bad IPs), custom rules (IP allow/deny, rate limiting per IP/URI, geo blocking). Actions: ALLOW, BLOCK (403), COUNT (monitoring mode). Rate limit: block IPs exceeding N requests/5min. Regional WAF attaches to ALB for non-CloudFront traffic.

Q9. What is Lambda@Edge and when is it used with Java backends?

Lambda@Edge runs Node.js/Python functions at CloudFront edge nodes (not Java -- too heavy). Use for: origin request modification (add auth headers before Java backend), response transformation (add security headers to all responses), A/B testing (modify origin based on cookie), geo-based routing (route to nearest Java regional deployment). Limitations: no VPC access, 5s timeout (viewer), 30s (origin), 1MB code size.

Q10. How do you optimise Lambda memory for Java Spring Boot?

Lambda memory also determines CPU allocation (1769MB = 1 vCPU). Java Spring Boot needs minimum 512MB (basic apps) to 2048MB (complex Spring apps). Higher memory = more CPU = faster init and execution. Cost: memory x duration. Optimal: use

AWS Lambda Power Tuning (open-source) to benchmark at 512, 1024, 1792, 3008MB -- find minimum cost point. SnapStart reduces init time making lower memory viable.